빅데이터 분석 & 자동화 시스템 구축 과정 (1000시간) · 최종 프로젝트 발표

HOMEPICK

공간을 공간답게 — 홈픽

국토교통부 실거래 빅데이터를 AI가 학습해
내 예산과 조건에 맞는 아파트 단지를 찾아 주는 부동산 플랫폼



팀원 길길동 (조장) · 박미금 · 김서연 · 홍길동

128,941	R² 0.938	2분 30초	2곳
건 실거래 학습	AI 적정가 예측력	원클릭 재학습	자동 배포 서버

발표 순서

01 **프로젝트 개요**

팀 소개 · 추진 배경 · 목표

발표 길길동

02 **시스템 설계**

아키텍처 · 기술 스택

발표 길길동 · 홍길동

03 **빅데이터 수집 & AI 분석**

크롤링 · 전처리 · RandomForest · KMeans

발표 박미금

04 **자동화 & 백엔드**

원클릭 재학습 · 회원 인증 · DB/API

발표 홍길동

05 **화면 시연**

AI 실거래 단지 추천 · 회원 화면

발표 김서연

06 **배포 · 트러블슈팅 · 성과**

CI/CD · 문제 해결 · 향후 계획

발표 홍길동 · 김서연 · 길길동

팀 소개 및 역할 분담



길길동 조장

조장 · PM · 아키텍처

- 일정 · 요구사항 관리
- 시스템 아키텍처 설계
- 산출물 문서 총괄
- 최종 통합 · 발표 총괄



박미금

데이터 수집 · AI 모델

- 공공데이터 API 크롤러 개발
- 전처리 · 이상치 제거 · EDA
- RandomForest 적정가 모델
- KMeans 단지 유형 · 인사이트



홍길동

백엔드 · 자동화 · 배포

- 회원가입 · 로그인 · BCrypt
- Firebase · 소셜 로그인
- Actions 재학습 자동화
- Render · AWS EC2 CI/CD



김서연

프론트엔드 · UI/UX

- 화면 설계 · 퍼블리싱
- AI 추천 · 분석 리포트 화면
- Plotly 시각화 · 반응형
- 공통 헤더 · 푸터 · 파비콘

추진 배경 — 왜 '실거래 데이터'인가?

01 호가 ≠ 실제 거래가

부동산 플랫폼의 매물 가격은 집주인이 부르는 '호가'로, 실제 거래 가격과 차이가 큽니다. 내 예산으로 실제 어떤 단지를 살 수 있었는지 알기 어렵습니다.

02 흩어진 공공 빅데이터

국토교통부 실거래 데이터는 공개되어 있지만 지역 · 월별 API로 쪼개져 있어 일반 사용자가 모아서 비교 · 분석하기 어렵습니다.

03 '적정 가격' 기준의 부재

같은 지역 · 같은 면적이라도 가격 차이가 큰데, 이 거래가 비싼지 싼지 판단할 객관적인 기준이 없습니다.

해결 아이디어 공공 실거래 빅데이터를 자동으로 수집 · 학습해 AI 적정가 와 실제 거래 기록 으로 단지를 추천하자

프로젝트 목표와 핵심 기능

수집

공공 API 크롤링

수도권 83개 시·군·구
최근 6개월 실거래

분석

AI 학습

RandomForest 적정가
KMeans 단지 유형

서비스

AI 단지 추천

예산 · 방 수 · 지역
맞춤 순위 + 근거

시각화

분석 리포트

월별 추이 · 가격 요인
시·군·구 순위 (Plotly)

자동화

원클릭 재학습

버튼 한 번에 수집→학습
→반영 (약 2분 30초)

함께 구현한 서비스 기능

4단계 회원가입 (Firebase 휴대폰 인증)

카카오 · 네이버 · 구글 소셜 로그인

BCrypt 비밀번호 암호화

Q&A 게시판

Render · EC2 자동 배포

※ 실거래 데이터는 이미 거래가 끝난 기록이며, 현재 판매 중인 매물과 다를 수 있음을 화면에 명시했습니다.

시스템 아키텍처

사용자
PC · 모바일

운영 서버 (같은 코드 · 같은 저장소)

Render
homepick.onrender.com
Docker 컨테이너

AWS EC2 (Ubuntu)
homepickkr.duckdns.org
nginx + Let's Encrypt SSL
↓
Docker · Spring Boot :8080

GitHub

저장소 main
소스 + apartments.json

Actions · deploy.yml
빌드 · 테스트 · EC2 배포

Actions · ai-data-refresh.yml
크롤링 · 전처리 · 학습

외부 서비스

TiDB Cloud
member · qna

Firebase Auth
휴대폰 SMS 인증

OAuth2
카카오 · 네이버 · 구글

국토교통부 실거래가 API

① Render · EC2는 같은 저장소의 코드를 각각 자동 배포
② AI 학습은 GitHub Actions에서 1회 실행 → 결과 JSON 하나를 두 서버가 공유
③ 데이터만 바뀌면 재배포 없이 서버가 GitHub에서 최신 JSON을 직접 수신

기술 스택

Backend	Java 21 · Spring Boot 4.1 Spring MVC · JdbcTemplate Spring Security (BCrypt · OAuth2)
Frontend	Thymeleaf · Bootstrap 5.3 Vanilla JS · Font Awesome plotly.js 4.1
Data · AI	Python 3.11 · requests · pandas scikit-learn (RandomForest · KMeans) Plotly 7.1 · matplotlib
Database · 인증	TiDB Cloud (MySQL 호환) Firebase Phone Auth Kakao · Naver · Google OAuth2
자동화 · CI/CD	GitHub Actions (빌드 · 테스트 · 배포 · AI 재학습) Gradle · Docker
Infra	Render (Docker) AWS EC2 · nginx · Let's Encrypt DuckDNS

빅데이터 수집 — 공공데이터 API 크롤링

83 개 지역

서울 25 · 인천 11 · 경기 47

6 개월

2026.04 ~ 2026.09

498 회

지역 x 월 API 요청

128,941 건

실거래 · 18,129개 단지

수집 방식

- 출처: 국토교통부 아파트 매매 실거래가 API (data.go.kr)
- 법정동 코드 5자리 기준, 행정구역 개편(인천 · 화성) 반영
- 8건 동시 병렬 요청 (ThreadPool) + 실패 시 1·2·4초 재시도
- 실패 비율이 높으면 중단하고 원인을 로그 · 화면에 출력

병렬 크롤링 성능 개선

개선 전 (순차 요청)

28분+

→

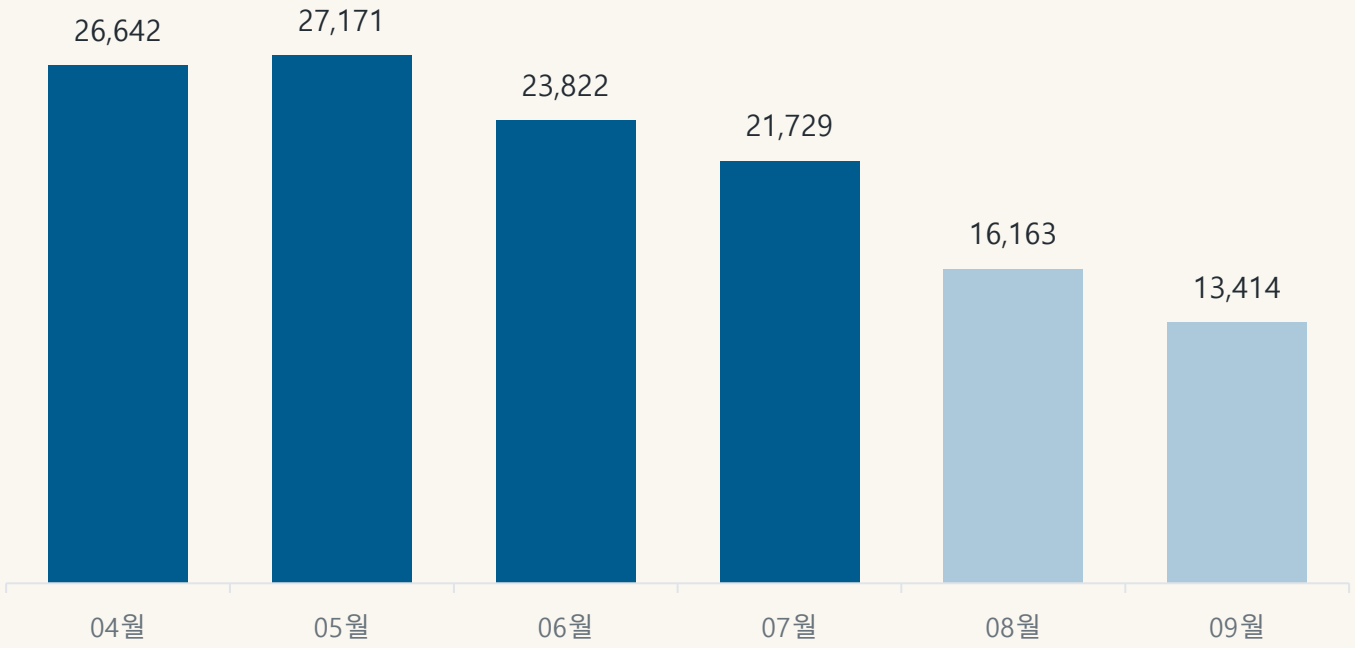
개선 후 (8건 병렬)

27초

API 응답이 건당 0.7~14.6초로 느렸던 날, 498건을 하나씩 보내면서 재시도 대기까지 쌓인 것이 원인이었습니다.

전처리와 탐색적 데이터 분석 (EDA)

수도권 월별 실거래 건수



인사이트 중위 거래가 6.3억(5월) → 5.8억(7월)로 하락, 거래가 가장 활발했던 달은 5월(27,171건).
최근 2개월(연한 막대)은 실거래 신고 기한(30일) 때문에 아직 덜 집계된 값입니다.

전처리 단계

1 정제
금액 쉼표 · 공백 제거, 숫자 변환

2 이상치 제거
면적 · 가격 극단값, 해제된 거래 제외

3 파생 변수
평당가, 건물 연식, 계약일

4 방 수 추정
전용면적 40·60·85㎡ 기준 1~4방+

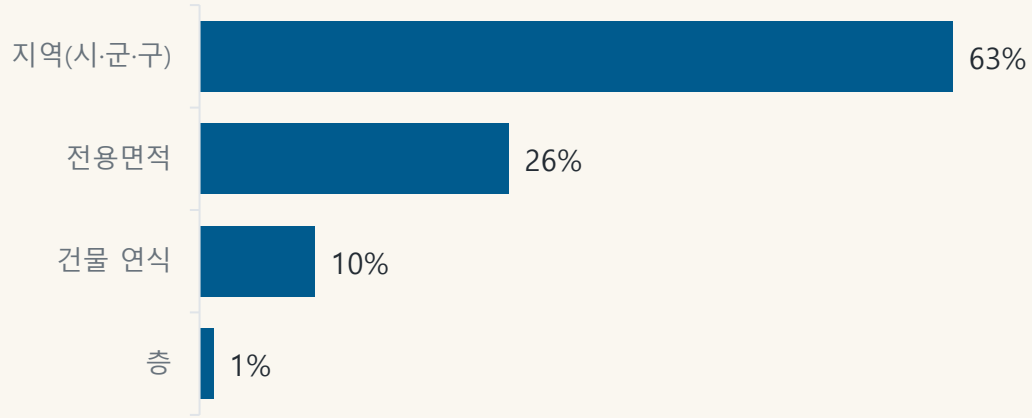
AI 모델 ① RandomForest — 적정 거래가 예측

0.938	7,333만원	33%
결정계수 R^2 (예측력)	평균 절대 오차 (MAE)	저평가 거래 비율 (적정가보다 5%↓)

모델 구성

- 입력: 전용면적 · 층 · 건물 연식 + 지역(One-Hot)
- RandomForestRegressor (트리 200개, 최소 리프 3)
- 저평가 점수(valueScore) = AI 적정가 ÷ 실제 거래가
- 1보다 크면 적정가보다 싸게 거래된 단지

AI가 본 가격 결정 요인 (Feature Importance)



→ 가격은 지역(63%) 과 전용면적(26%) 이 대부분 결정하고, 층은 거의 영향이 없었습니다.

AI 모델 ② KMeans 단지 유형 & 추천 점수

AI 추천 점수 (100점)

50% 조건 근접도

예산 · 면적에 가까울수록

35% 저평가 점수

AI 적정가보다 싸게 거래될수록

15% 최근 거래

최근에 거래된 단지일수록

필터: 희망 지역 → 예산 이하 → 방 수 일치

KMeans 군집 → 사람이 읽는 단지 유형 이름

단지 유형	중위 거래가	면적 · 연식	대표 지역
구축·대형·고가	35억	131㎡ · 24년	서울 강남구
준신축·중형·고가	18.2억	85㎡ · 11년	서울 서초구
구축·중형·고가	15.4억	84㎡ · 30년	경기 성남시 분당구
구축·대형·중간가	8.3억	122㎡ · 23년	경기 용인시 수지구
신축·중소형·중간가	5.7억	75㎡ · 7년	경기 평택시

가격 · 면적 · 연식이 비슷한 단지끼리 자동으로 묶고, 군집 중심값으로 '신축 / 구축', '중소형 / 대형', '중간가 / 고가' 이름을 붙였습니다.

원클릭 AI 재학습 자동화 (GitHub Actions)

클릭
로그인 사용자
재학습 버튼

준비
35초

크롤링
27초

전처리
4초

AI 학습
70초

커밋
2초

서버 반영
Render · EC2
동시 적용

전체 소요 시간

약 2분 30초

Python 서버 없이 GitHub Actions가
크롤링 → 학습 → 결과 커밋까지
전부 자동 실행

진행 상태 실시간 표시

- 5초마다 GitHub 실행 단계 조회
- 경과 시간 · 진행 막대 표시
- 단계가 뒤로 가지 않게 보정
- 실패 시 실패 단계와 원인 출력

안전 장치

- 로그인 사용자만 실행 (비로그인 401)
- 10분 내 재요청 · 중복 실행 차단
- 크롤링 20분 · 전체 30분 제한
- GitHub 장애 시 내장 JSON 사용

회원가입 · 로그인 · 보안

4단계 회원가입

1. 약관 동의	필수 2 · 선택 2 전체 동의 연동
2. 기본 정보	이름 · 생년월일 성별
3. 휴대폰 인증	Firebase SMS 6자리 · 3분
4. 계정 생성	아이디 · 비밀번호 서버 재검증

보안 처리

- 비밀번호 BCrypt 해시 저장 (평문 저장 없음)
- Firebase 본인인증 UID 없이는 가입 불가
- 화면 · 서버 같은 규칙으로 이중 검증 (400 · 409 응답)
- API 키 · DB 비밀번호는 .env · 환경변수 · GitHub Secrets로 분리

소셜 로그인 (OAuth2)

카카오 로그인 | **네이버 로그인** | **구글 로그인**

- 처음 로그인 → 자동 회원가입
- 기존 회원 → 이름 · 이메일 갱신
- Spring Security OAuth2 Client
- nginx 뒤에서도 https 콜백 주소 생성

데이터베이스 · API 설계

주요 API

Method	URL	설명
POST	/api/members/register	회원가입 (JSON)
POST	/member/login	아이디 로그인
GET	/oauth2/authorization/{sns}	소셜 로그인
GET	/api/ai-data/apartments	AI 학습 데이터 (gzip)
GET	/api/ai-data/status	재학습 진행 상태
POST	/api/ai-data/refresh	AI 재학습 (로그인)
GET	/qna	Q&A 목록
GET	/healthz	헬스 체크

TiDB 테이블

member (회원)	설명
user_id UK	로그인 아이디
password_hash	BCrypt 해시
name · birth · gender · phone	기본 정보
firebase_uid	휴대폰 본인인증
sns_type · sns_id UK	소셜 로그인 식별

qna (Q&A)	설명
title · content · writer	제목 · 내용 · 작성자
status · created_at	상태 · 작성일

AI 실거래 단지 추천 화면

① 실거래 기준 안내

'판매 중인 매물이 아닌 거래 완료 기록' 명시

② 조건 입력 바

예산 · 방 수 · 지역 → AI 단지 추천 받기

③ 재학습 버튼 · 진행 표시

로그인 사용자 전용, 단계 · 경과 시간

④ AI 실거래 분석 리포트

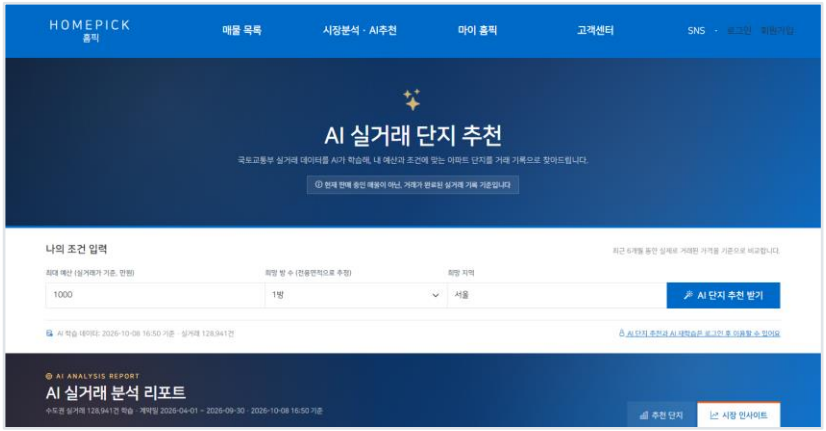
추천 단지 / 시장 인사이트 탭

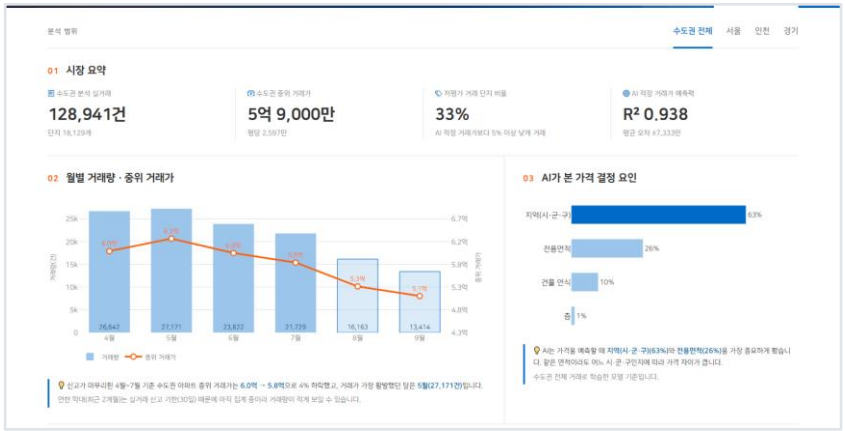
⑤ 시장 인사이트

수도권 · 서울 · 인천 · 경기 범위 탭, 시장 요약 지표

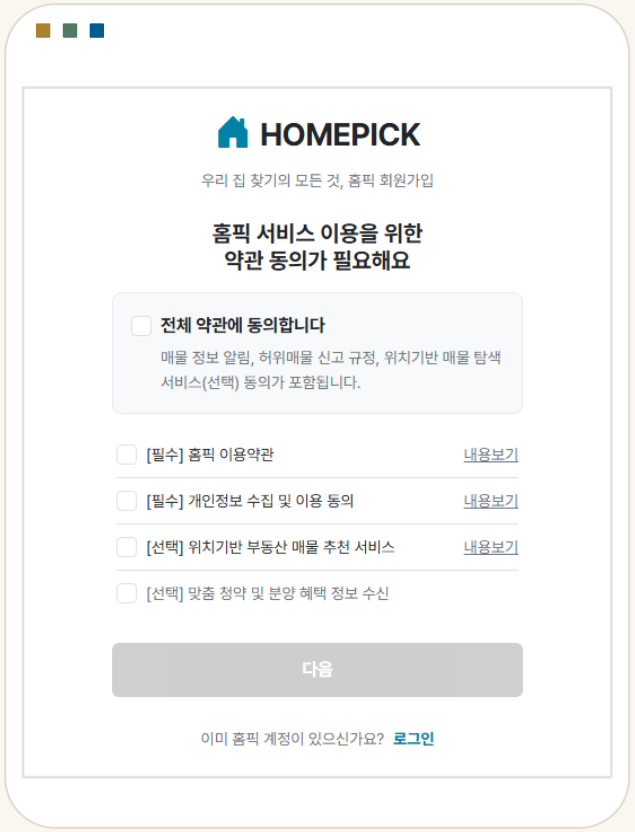
⑥ Plotly 그래프

Python에서 만든 그래프를 plotly.js로 표시



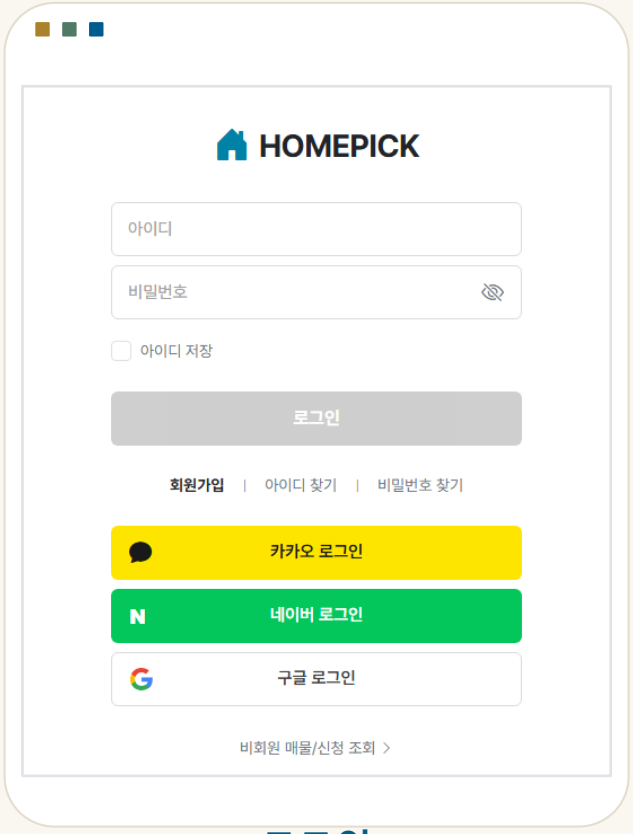


회원가입 · 로그인 · Q&A 화면



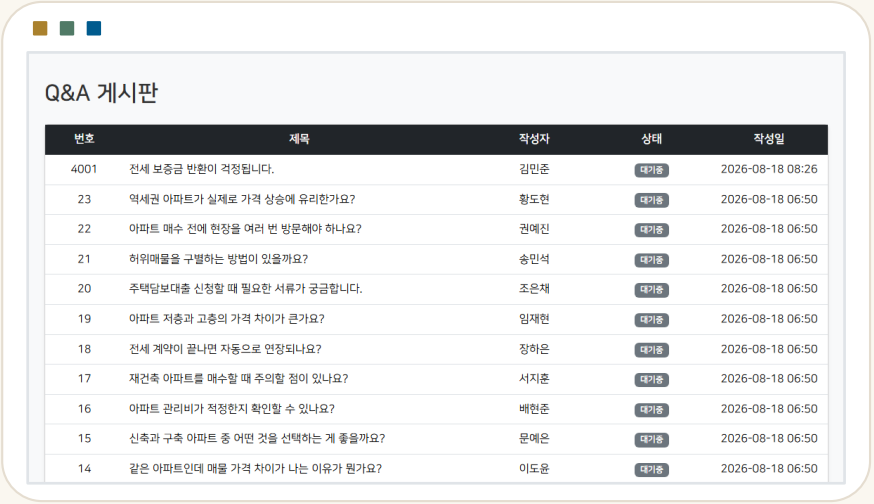
회원가입 (1단계 약관)

전체 동의 ↔ 개별 4개 연동



로그인

아이디 로그인 + 소셜 로그인 3종



Q&A 게시판

TiDB qna 테이블 목록

CI/CD 배포 자동화

git push main

Render

자동 감지 → Dockerfile 빌드 → 배포 완료

AWS EC2 (GitHub Actions)

빌드 · 테스트 → Docker 검사 → Secrets → env → SSH 접속 → Docker 교체 → 응답 확인

build 작업 (GitHub 서버)

deploy 작업 (EC2에서 실행 · 실패 시 컨테이너 로그 출력)

포인트

- 앱 환경변수를 GitHub Secrets로 관리
- 컨테이너는 127.0.0.1:8080, nginx로만 접근
- AI 데이터 커밋은 재배포 생략

결과 코드를 main에 올리기만 하면 테스트를 통과한 버전이 Render와 EC2 두 곳에 자동 배포됩니다.

트러블슈팅 — 문제를 이렇게 해결했습니다

문제	원인	해결	결과
AI 재학습 중 크롤링이 27분 넘게 끝나지 않음	API 응답 지연(건당 최대 14.6초) + 498건 순차 요청	8건 병렬 요청, 재시도 대기 단축, 단계별 시간 제한	크롤링 27초 · 전체 2분 30초
진행 표시가 '전처리 → 크롤링'으로 되돌아감	GitHub 단계 정보가 순간적으로 어긋나게 조회됨	가장 많이 진행된 단계를 기억해 뒤로 가지 않게 보정	단계 순서 항상 일치
GitHub Actions 빌드(테스트) 실패	CI 환경에 소셜 로그인 client-id가 없어 앱 시작 실패	테스트에서만 더미 client-id 지정 (운영 설정 유지)	CI 빌드 · 테스트 통과
EC2 자동 배포가 SSH 연결에서 실패	인스턴스 재시작으로 퍼블릭 IP 변경, Secret에는 예전 IP	EC2_HOST를 도메인으로 변경, IP 등록 시 경고 출력	IP가 바뀌어도 배포 유지
'매물 추천'이 현재 매물로 오해될 수 있음	데이터는 거래가 끝난 실거래 기록	페이지 · 문구를 '실거래 단지 추천'으로 전면 수정	데이터 의미를 정확히 전달

→ 모든 문제와 해결 과정은 저장소 docs/ 폴더에 문서로 기록해 팀 전체가 공유했습니다.

프로젝트 성과와 향후 계획

성과

12.9 만 건
수도권 실거래 학습

0.938 R^2
AI 적정가 예측력

98 % 단축
크롤링 시간 (28분+ → 27초)

2 곳
자동 배포 (Render · EC2)

수집 → 분석 → 서비스 → 시각화 → 자동화를 하나의 파이프라인으로 연결해 실제 운영 중

향후 계획

- 전월세 실거래 · 전국 단위로 데이터 확장
- 시계열 예측 모델로 향후 가격 추이 예측
- XGBoost 등 모델 비교로 예측 정확도 개선
- 스케줄러로 매일 자동 재학습
- 지도 기반 단지 조회 · 관심 단지 알림
- Q&A 글쓰기 · 마이페이지 등 미구현 메뉴 완성

Q & A

감사합니다

HOMEPICK — 빅데이터 분석 & 자동화 시스템 구축 과정 최종 프로젝트

길길동 (조장) · 박미금 · 김서연 · 홍길동

https://homepick.onrender.com · https://homepickkr.duckdns.org
github.com/fsclass-n/homepick_lo